

Deployment

Render

`render.yaml` defines a Python web service:

```
services:
  - type: web
    name: ctc-mkwii-api
    env: python
    buildCommand: cd backend && pip install -r requirements.txt
    startCommand: cd backend && gunicorn app:app
    envVars:
      - key: PYTHON_VERSION
        value: 3.11
```

This deploys the backend API only. The frontend fallback API URL points at:

```
https://ctcmkwiistatswebsite.onrender.com
```

Railway

`railway.json` says Railway should build with the Dockerfile:

```
{
  "build": {
    "builder": "dockerfile"
  }
}
```

The Dockerfile builds the frontend and then runs the backend in a Python runtime image.

Docker

Dockerfile:

1. Uses `node:22` to install frontend dependencies and run `npm run build`.
2. Uses `python:3.11-slim` for runtime.
3. Installs Node.js in the runtime image.
4. Copies `backend` and installs Python requirements.
5. Copies frontend build output from `/app/frontend/build` to `/app/frontend/build`.
6. Starts `/app/start.sh`.

Important: the Flask app currently does not serve the frontend build. As written, the Docker image builds frontend assets but `backend/app.py` only defines API routes. Unless another service serves `/app/frontend/build`, the container is effectively an API service with unused built frontend files.

GitHub Pages

`.github/workflows/deploy.yml`:

1. Runs on pushes to `main`.
2. Installs Node 18.
3. Runs `cd frontend && npm install`.
4. Runs `cd frontend && npm run build`.
5. Publishes `./frontend/build` to GitHub Pages.

This deploys frontend static assets only. Since no `VITE_API_URL` is set in this workflow, the built frontend uses the hardcoded Render API fallback.

The workflow uses `${{ secrets.GITHUB_TOKEN }}`, which is automatically provided by GitHub Actions. It is not a creator-owned secret you need to recover.

Local Development

Backend:

```
cd backend
pip install -r requirements.txt
python app.py
```

Frontend:

```
cd frontend
npm install
npm run dev
```

By default, the frontend code will still call the Render backend unless `VITE_API_URL` is set or the code is changed to use relative `/api` URLs.

Ports

- Flask debug default: `5000`
- Vite dev server: `3000`
- Docker/hosted Gunicorn: `${PORT:-5000}`
- Dockerfile exposes `8080`, but `start.sh` defaults to `5000` if `PORT` is not set.

That `EXPOSE 8080` vs default `5000` mismatch is worth cleaning up.